

DS-GA 1011 Assignment 2: Building a Transformer LM

Written Responses

Fall 2026

This writeup follows the uploaded Fall 2026 assignment handout. The implementation deliverables are contained in the accompanying code archive; the analytical results below use the architecture and accounting assumptions stated in the handout.

1. Transformer LM Resource Accounting

Throughout this section, let

V = vocab size, C = context length, N = number of layers,

$d = d_{\text{model}}$, h = number of heads, d_{ff} = FFN hidden size.

For the GPT-2 XL calculation, $V = 50,257$, $C = 1024$, $N = 48$, $d = 1600$, $h = 25$, and $d_{\text{ff}} = 6400$.

(a) Parameter count and model memory

There are two vocabulary-sized matrices: the input token embedding and the learned output embedding (LM head). Each contributes Vd parameters. Each Transformer block has three attention projections (W_Q, W_K, W_V), one attention output projection, three SwiGLU matrices, and two RMSNorm gain vectors, giving

$$4d^2 + 3dd_{\text{ff}} + 2d$$

parameters per block. The final RMSNorm contributes d more parameters. Hence

$$P = 2Vd + N(4d^2 + 3dd_{\text{ff}} + 2d) + d.$$

For GPT-2 XL,

$$P = 2(50,257)(1600) + 48(4(1600)^2 + 3(1600)(6400) + 2(1600)) + 1600 = 2,127,057,600.$$

At four bytes per float32 parameter, loading the model requires

$$4P = 8,508,230,400 \text{ bytes} \approx 8.51 \text{ GB} \approx 7.92 \text{ GiB}.$$

(b) Forward-pass matrix multiplies and FLOPs

The handout asks for the matrix multiplications only; elementwise operations such as RMSNorm, SiLU, RoPE, masking, and softmax are therefore not included in the matrix-multiply FLOP total. For one sequence of length C , the matrix multiplies are:

Operation	Description	FLOPs
Q, K, V projections	Three maps $(C \times d)(d \times d)$	$6Cd^2$
Attention scores $QK^\top$	Across all heads, $h(C \times d_h)(d_h \times C)$ with $d_h = d/h$	$2C^2d$
Attention weighted sum	Across all heads, $h(C \times C)(C \times d_h)$	$2C^2d$
Output projection	$(C \times d)(d \times d)$	$2Cd^2$
SwiGLU W_1	$(C \times d)(d \times d_{\text{ff}})$	$2Cdd_{\text{ff}}$
SwiGLU W_3	$(C \times d)(d \times d_{\text{ff}})$	$2Cdd_{\text{ff}}$
SwiGLU W_2	$(C \times d_{\text{ff}})(d_{\text{ff}} \times d)$	$2Cdd_{\text{ff}}$
LM head	$(C \times d)(d \times V)$	$2CdV$

Therefore, one Transformer block costs

$$F_{\text{block}} = 8Cd^2 + 4C^2d + 6Cdd_{\text{ff}},$$

and the whole model costs

$$F_{\text{fwd}} = N(8Cd^2 + 4C^2d + 6Cdd_{\text{ff}}) + 2CdV.$$

For GPT-2 XL, this gives

$$F_{\text{fwd}} = 4.5133365248 \times 10^{12} \text{ FLOPs} \approx 4.513 \text{ TFLOPs}$$

per sequence of 1024 tokens.

(c) Which components require the most FLOPs?

For GPT-2 XL, the FFN matrix multiplies account for about 66.91% of the total forward FLOPs, making them the dominant cost. The remaining major cost is the linear projections in attention, while the quadratic attention matrix multiplies and the vocabulary-sized LM head each contribute only a few percent at $C = 1024$.

(d) Scaling across GPT-2 sizes

For this comparison, the handout’s accounting assumption $d_{\text{ff}} = 4d$ is used, with the same vocabulary size and context length as above. The totals and component proportions are:

Model	Total (TFLOPs)	QKV	Attn. matmuls	Output proj.	FFN	LM head
GPT-2 small	0.349630	12.44%	11.06%	4.15%	49.75%	22.61%
GPT-2 medium	1.033110	14.97%	9.98%	4.99%	59.87%	10.20%
GPT-2 large	2.257755	16.05%	8.56%	5.35%	64.20%	5.84%
GPT-2 XL	4.513337	16.73%	7.14%	5.58%	66.91%	3.65%

As the model gets wider and deeper, the d^2 -scaling projection and FFN terms become a larger fraction of total compute, while the vocabulary projection becomes relatively smaller. The quadratic attention terms also become a smaller fraction at fixed context length because the width-dependent d^2 terms grow faster.

(e) Increasing the context length to 16,384

Keeping all GPT-2 XL model dimensions fixed and changing only C from 1024 to 16,384 gives

$$F_{\text{fwd}}(16,384) = 1.495227957248 \times 10^{14} \text{ FLOPs} \approx 149.52 \text{ TFLOPs},$$

which is about $33.13\times$ the cost at context length 1024. The attention score and value matrix multiplies now contribute about 55.15% of the total, while the FFN drops to about 32.32%, illustrating the quadratic C^2 scaling of attention versus the linear C scaling of the other major terms.

2. Learning Rate Tuning

Using the toy SGD example for 10 iterations, the behavior is:

Learning rate α	Observed behavior
10	Loss decreases steadily.
100	Loss decreases much more rapidly.
1000	Loss diverges / increases rapidly.

Thus, increasing the learning rate from 10 to 100 accelerates convergence in this toy problem, but 1000 is too large and causes unstable updates and divergence.

3. AdamW Resource Accounting

For this section, the handout explicitly asks us to assume $d_{\text{ff}} = 4d$ for the accounting calculation.

(a) Peak training memory

With $d_{\text{ff}} = 4d$, the parameter count is

$$P = 2Vd + N(16d^2 + 2d) + d.$$

The four memory components are as follows.

Parameters.

$$M_{\text{param}} = 4P \text{ bytes.}$$

Gradients. A gradient tensor has the same shape as the corresponding parameter tensor, so

$$M_{\text{grad}} = 4P \text{ bytes.}$$

Optimizer state. AdamW stores two float32 moment tensors, m and v , each of size P :

$$M_{\text{opt}} = 8P \text{ bytes.}$$

Activations. For one Transformer block, the activation count under the handout's simplified list is

$$\begin{aligned}
A_{\text{block}} = & 2BCd && \text{(two RMSNorm outputs)} \\
& + 3BCd && \text{(Q,K,V)} \\
& + 2BhC^2 && \text{(} QK^\top \text{ and softmax)} \\
& + BCd && \text{(weighted sum of V)} \\
& + BCd && \text{(output projection)} \\
& + 2BCd_{\text{ff}} && \text{(} W_1 \text{ and SiLU)} \\
& + BCd && \text{(} W_2 \text{).}
\end{aligned}$$

Using $d_{\text{ff}} = 4d$,

$$A_{\text{block}} = 16BCd + 2BhC^2.$$

Adding the final RMSNorm, output logits, and the logits used by cross-entropy gives

$$A = N(16BCd + 2BhC^2) + BCd + 2BCV.$$

Thus

$$M_{\text{act}} = 4A \text{ bytes.}$$

Combining all four pieces,

$$M_{\text{peak}} = 16P + 4A \text{ bytes.}$$

This separates the fixed model/optimizer cost, which depends only on P , from the batch-dependent activation cost.

(b) GPT-2 XL on an 80 GB device

Substituting $V = 50,257$, $C = 1024$, $N = 48$, $d = 1600$, and $h = 25$ gives

$$P = 2,127,057,600,$$

and

$$M_{\text{peak}}(B) = 15.517753344 B + 34.0329216 \text{ GB}$$

when memory is expressed in decimal GB. Using the usual binary device-memory convention ($1 \text{ GiB} = 2^{30}$ bytes), the same expression is

$$M_{\text{peak}}(B) = 14.452034 B + 31.695628 \text{ GiB}.$$

Under an 80 GiB budget,

$$14.452034B + 31.695628 \leq 80,$$

so the maximum integer batch size is

$$B_{\text{max}} = 3.$$

(c) FLOPs for one AdamW step

The AdamW update is elementwise in the parameters and their two moment buffers, so its compute cost is linear in the parameter count:

$$F_{\text{AdamW}} = \Theta(P).$$

With the conventional simplified accounting of roughly 10 scalar operations per parameter for the moment updates, bias correction, parameter update, and decoupled weight decay,

$$F_{\text{AdamW}} \approx 10P$$

and, for GPT-2 XL,

$$F_{\text{AdamW}} \approx 2.1270576 \times 10^{10} \text{ FLOPs} \approx 21.27 \text{ GFLOPs.}$$

(d) A100 training-time estimate

From part 1(b), the GPT-2 XL forward pass for one 1024-token sequence costs

$$F_{\text{fwd}} = 4.5133365248 \times 10^{12} \text{ FLOPs.}$$

For batch size $B = 1024$, the forward cost per step is

$$1024F_{\text{fwd}} = 4.6216566013952 \times 10^{15}.$$

The handout asks us to take the backward pass to be twice the forward cost, so the model computation per training step is

$$F_{\text{train step}} = F_{\text{fwd}} + 2F_{\text{fwd}} = 3(1024)F_{\text{fwd}} = 1.38649698041856 \times 10^{16} \text{ FLOPs.}$$

The AdamW optimizer contribution is negligible by comparison. Over 400,000 steps,

$$F_{\text{total}} \approx 5.54599 \times 10^{21} \text{ FLOPs.}$$

At 50% MFU on an A100 with a 19.5 TFLOP/s FP32 peak, the effective throughput is

$$0.5(19.5 \times 10^{12}) = 9.75 \times 10^{12} \text{ FLOP/s.}$$

Therefore,

$$t \approx \frac{5.54599 \times 10^{21}}{9.75 \times 10^{12}} \approx 5.6878 \times 10^8 \text{ s} \approx 6584 \text{ days} \approx 18.04 \text{ years.}$$

Thus the requested estimate is

about 18.0 years on one A100.

4. First Training Run

The implementation combines the custom Transformer LM, cross-entropy, AdamW, data loader, and (optionally) the cosine learning-rate schedule into a single TinyStories training loop. The accompanying Colab notebook records the loss versus training step and produces the required learning-curve plot.

The supplied local execution environment did not contain the handout-provided TinyStories binary data, so I do not report fabricated loss values or claim an empirical curve that was not actually observed. The notebook is configured to load ‘data/tinystories_valid.bin’ via ‘load_memmap_dataset’, train the small model suggested by the handout, and display the required downward-trending loss curve once those files are available.

Assignment-result note

Before submitting the final PDF, run the TinyStories training cell in the accompanying Colab notebook and insert the resulting learning-curve figure here (or export the notebook result alongside this writeup). The implementation and plotting code are already prepared in the notebook.

5. Decoding

The decoding implementation lives in

`student/generation.py`

and is exposed through the package-level function ‘generate’. It supports a user prompt, a maximum number of generated tokens, temperature scaling, top- p (nucleus) sampling, and early termination on an end-of-text token; the accompanying notebook demonstrates the generation path.

Acknowledgements

Certain code and text in this submission were developed with AI assistance.